Prikshit Kumar Angra, Lovely Professional University

Unit 09: Working with Streams and Files

CONTENTS

Objectives

After studying this unit, you will be able to:

- Demonstrate the opening a file
- Understand C++ Stream classes
- Explain the processing and closing a file
- Discuss the detection of end of file

Introduction

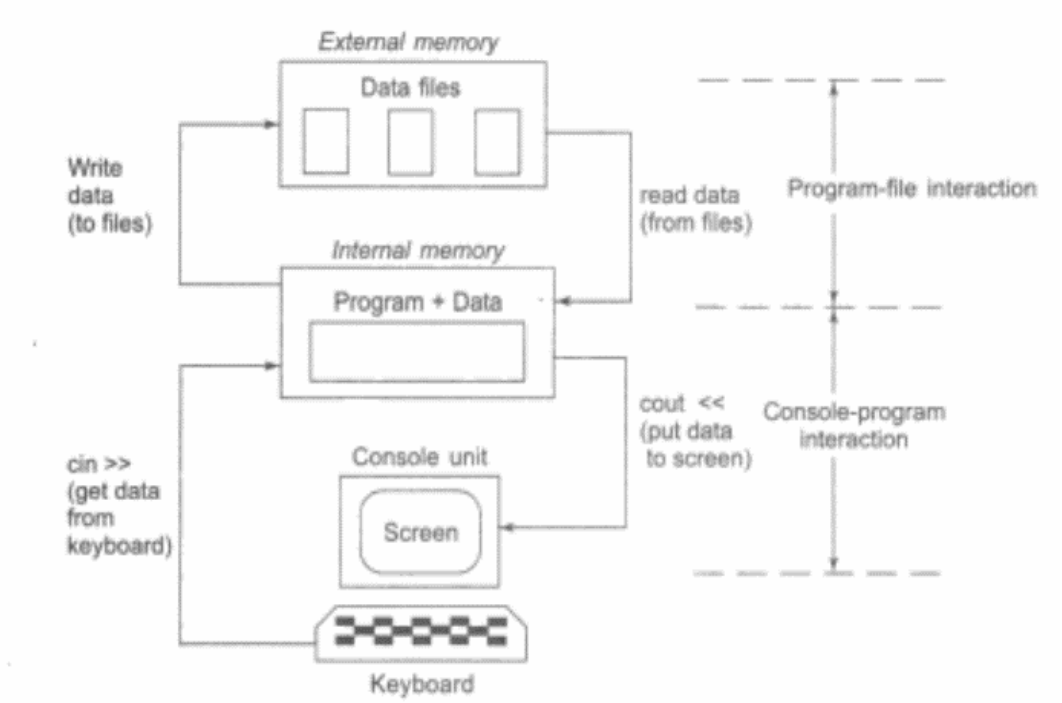
Many real-life problems handle large volumes of data and, in such situations, we need to use some devices such as floppy disk or hard disk to store the data. The data is stored in these devices using the concept of files. A file is a collection of related data stored in a particular area on the disk. Programs can be designed to perform the read and write operations on these files.

A program typically involves either or both of the following kinds of data communication:

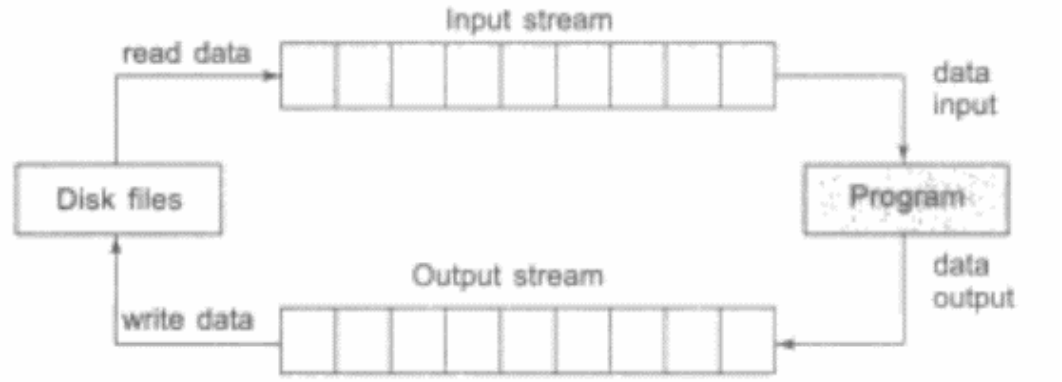
- Data transfer between control unit and the program.
- Data transfer between the program and a disk file.

We have already discussed the technique of handling data communication between the console unit and the program. In this chapter, we will discuss various methods available for storing and retrieving the data from files.

This is illustrated in figure.



C++'s I/O system deals with file operations that are quite similar to console input and output activities. As an interface between the applications and the files, it employs file streams. The input stream is the one that provides data to the programme, while the output stream is the one that receives data from the programme. To put it another way, the input stream pulls (or reads) data from the file, whereas the output stream writes data to the file. Figure shows how this works.



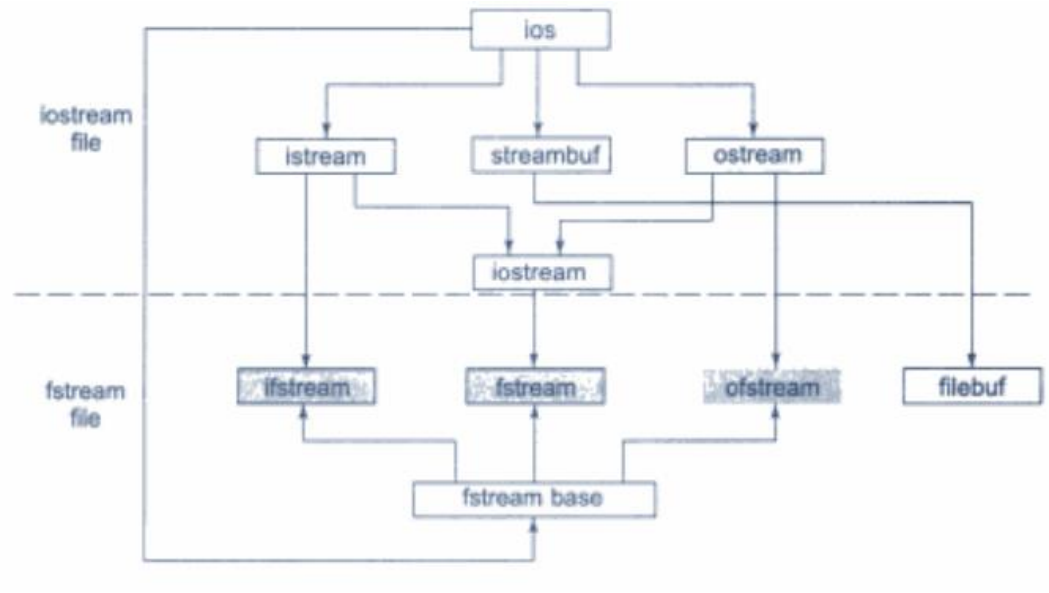
The input operation entails establishing an input stream and connecting it to the programme and the input file. Similarly, setting up an output stream with the appropriate linkages to the programme and the output file is part of the output procedure.

Notes

- Stream classes in C++ are used to input and output operations on files and i/ o devices. These classes have specific features and to handle input and output of the program.
- The **iostream.h** library holds all the stream classes in the C++ programming language.

9.1 Classes for File Stream Operations

The file handling techniques are defined by a set of classes in C++'s I/O system. Ifstream, ofstream, and (stream) are a few examples. As illustrated in Figure, these classes are generated from fstreambase and the matching tostream class. These classes, designed to manage disc files, are declared in (stream, and as a result, every programme that uses files must include this file.



Details of file stream classes listed in following table.

Did you know?

What is Stream in C++?

- A stream is nothing but a flow of data.
- In the object-oriented programming, the streams are controlled using the classes. The operations with the files mainly consist of two types. They are read and write.
- C++ provides various classes, to perform these operations.
- Each stream is associated with a particular class, which contains member functions and definitions for dealing with that particular kind of data flow.
- The stream that supplies data to the program in known as an input stream. It reads the data from the file and hands it over to the program.
- The stream that receives data from the program is known as an output stream. It writes the received data to the file.

Benefits of Stream Classes

- The ios class: The ios class is responsible for providing all input and output facilities to all other stream classes.
- The istream class: This class is responsible for handling input stream. It provides number of function for handling chars, strings and objects such as get, getline, read, ignore, putback etc.

Classes for File Stream Operation

- Files are dealt mainly by using three classes fstream, ifstream, ofstream.

ofstream: This Stream class signifies the output file stream and is applied to create files for writing information to files

ifstream: This Stream class signifies the input file stream and is applied for reading information from files

fstream: This Stream class can be used for both read and write from/to files.

File Stream Classes

Class	Purpose
filebuf	Its purpose is to set the file buffers to read and write. Contains **Openprot** constant used in the **open()** of file stream classes. Also contains **close()** and **open()** as members.
fstreambase	Provides operations common to file streams Serves as a base for fstream, ifstream and ofstream class. Contains open() and close() functions.
ifstream	Provides input operations. Contains open° with default input mode. Inherits the functions get(), getline(),read(), seekg(), tellg() functions from istream.
ofstream	Provides output operations. Contains open() with default output mode. Inherits put(), seekp(), tellp() and write() functions from onstream.
fstream	Provides support for simultaneous input and output operations. Contains open with default input mode. Inherits all the functions from istream and ostream classes through iostream.

9.2 Creating A File

We create a file by specifying new path of the file and mode of operation. Operations can be reading, writing, appending and truncating.

Syntax

FilePointer.open("Path",ios::mode);

Example

```
#include<iostream>
#include <fstream>
using namespace std;
int main()
{
   fstream st;
   st.open("test.txt",ios::out);
   if(!st)
   {
      cout<<"File creation failed";
   }
   else
   {
```

```
    cout<<"New file created";
    st.close();
  }
  return 0;
}
```

Output

```
New file created
Process returned 0 (0x0)   execution time : 0.013 s
Press any key to continue.
```

9.3 Opening a File

The example programs listed above are indeed very simple. A data file can be opened in a program in many ways. These methods are described below.

ifstream filename("filename <with path>"); Or ofstream filename("filename <with path>");

This is the form of opening an input file stream and attaching the file "filename with path" in one single step. This statement accomplishes a number of actions in one go:

1. creates an input or output file stream

2. Looks for the specified file in the file system

3. Attaches the file to the stream if the specified file is found otherwise returns a NULL value.

The pointer is set to the initial location if the file was successfully joined to the stream. The object's name can be used to access the created file stream. If a path is supplied, the requested file is searched in that directory; otherwise, the file is only searched in the current directory. If the file isn't found, a new one is generated in case it's being used for output. If the requested file is identified while opening a file for output, it is truncated before being opened, resulting in the loss of everything in the file previously. It is important to guarantee that the application does not mistakenly overwrite a file. If the file is not discovered, a NULL value is given, which we can check to make sure we aren't reading a file that isn't there. If we try to read a file that isn't there, we'll get an error in the application.

ifstream filename;

filename.open("file name <with path>");

In this approach the input stream - filename - is created but no specific file is attached to the stream just created. Once the stream has been created a file can be attached to the stream using open() member function of the class ifstream or ofstream as is exemplified by the following program snippet which defines a function to read an input file.

```
#include <fstream.h>
void read(ifstream &ifstr)                    // file streams can be passed to functions
{
char ch;
while(!ifstr.eof())
{
ifstr.get(ch);
cout << ch;
```

```
}
cout << endl << " – – – –" << endl; Notes
}
void main()
{
ifstream filename("data1.dat");
read(filename);
filename.close();
filename.open("data2.dat");
read(filename);
filename.close();
}
```

ifstream filename(char *fname, int open_mode);

The ifstream function Object() { [native code] } accepts two parameters in this form: a filename and the mode in which the input file should be read. C++ has a variety of input file opening modes, each of which provides distinct forms of reading control over the opened file. In C++, the file opening modes are represented by an enumerated type called ios. Below is a list of the various file opening modes.

The ifstream function Object() { [native code] } accepts two parameters in this form: a filename and the mode in which the input file should be read. C++ has a variety of input file opening modes, each of which provides distinct forms of reading control over the opened file. In C++, the file opening modes are represented by an enumerated type called ios. Below is a list of the various file opening modes.

Opening Mode	Description
ios::in	Open file in input mode for reading
ios::out	Open file in output mode for writing
ios::app	Open file in output mode for writing the new content at the end of the file without removing the previous contents of the file.
ios::ate	Open file in output mode for writing the new content at the current position of the pointer without removing the previous contents of the file.
ios::trunc	Open file in output mode for writing the new content at the beginning of the file removing the previous contents of the file.
ios::nocreate	The file is not created. The operation takes place on existing file. If the file is not found an error occurs.
ios::noreplace	The existing file is not overwritten. The operation takes place on existing file. If the file is not found an error occurs.
ios::binary	Opens the file in binary mode reading not a character but reading/writing whatever binary value is stored in the file.

9.4 Opening and Closing File

If we want to use a disk file, we need to decide the following things about the file and its intended use:

1. Suitable name for the file
2. Data type and structure
3. Purpose

4. Opening Method

The filename is a string of characters that make up a valid filename for the operating system. It may contain two parts, a primary, name and an optional period with extension. Examples:

Input.data

Test.doc

INVENT.ORY

student

salary

OUTPUT

As stated earlier, for opening a file, we must first create a file stream and then link it to the filename. A file stream can be defined using the classes ifstream, ofstream, and factream that are contained in the header file fttream. The class to be used depends upon the purpose, that is, whether we want to read data from the file or write data to it. A file can be opened in two ways:

As stated earlier, for opening a file, we must first create a file stream and then link it to the filename. A file stream can be defined using the classes ifstream, ofstream, and fstream that are contained in the header file fstream. The class to be used depends upon the purpose, that is, whether we want to read data from the file or write data to it. A file can be opened in two ways:

1. Using the constructor function of the class.
2. Using the member function open() of the class.

The first method is useful when we use only one file in the stream. The second method is used when we want to manage multiple files using one stream.

Opening File Using Constructor

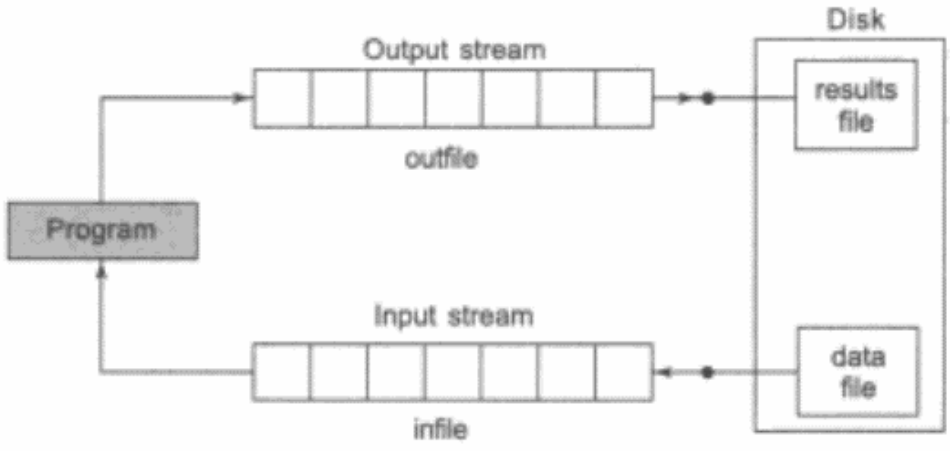
We know that a constructor is used to initialize an object while it is being created. Here, a filename is used to initialize the file stream object. This involves the following steps:

1. Create a file stream object to manage the stream using the appropriate class. That is to say, the class ofstream is used to create the output stream and the class ifstream to create the input stream.
2. Initialize the file object with the desired filename.

For example, the following statement opens a file named "results" for output:

ofstream outfile("results"); // output only

This creates outfile as an ofstream object that manages the output stream. This object can be any valid C++ name such as o_file, myfile or Pout. This statement also opens the file results and attaches it to the output stream outfile. This is illustrated in Figure.



Similarly, the following statement declares infile as an ifstream object and attaches it to the file data for reading (input).

ifstream infile("data"); // input only

The program may contain statements like:

outfile « "TOTAL.;

outfile « sum;

infile » number;

infile >> string;

Lab Exercise

```
#include <iostream>
#include <fstream>
using namespace std;
int main()
{
   ofstream of("demo.txt");
   of<< "Writing to file using fstream constructor!" << endl;
   of.close ();
   return 0;
}
```

Notes: The constructors of stream classes (ifstream, ofstream, or fstream) are used to initialize file stream objects with the filenames passed to them.

Opening File Using open()

As stated earlier, the function open() can be used to open multiple files that use the same stream object. For example, we may want to process a set of files sequentially. In such cases, we may create a single stream object and use it to open each file in turn. This is done as follows:

```
file-stream-class stream-object;
stream-object.open ("filename");
```

Example:

```
ofstream outfile;           // Create stream (for output)
outfile.open("DATA1");      // Connect stream to DATA1
.....
.....
outfile.close();            // Disconnect stream from DATA1
outfile.open("DATA2);       // Connect stream to DATA2
.....
.....
outfile.close();            // Disconnect stream from DATA2
.....
.....
```


Notes

- If the situation requires simultaneous processing of two files, then you need to create a separate stream for each file.


Lab Exercise

```
#include <iostream>
#include <fstream>
using namespace std;
int main()
{
string text;
ifstream ReadFile("a.txt");
while (getline (ReadFile, text)) {
 cout << text;
}
ReadFile.close();
return 0;
}
```

9.5 Detection end-of-file

Detection of the end-of-file condition is necessary for preventing any further attempt to read data from the file. This was illustrated in Program by using the statement.

while(fin)

An ifstream object, such as fin, returns a value of 0 if any error occurs in the file operation including the end-of-file condition. Thus, the while loop terminates when fin returns a value of zero on reaching the end-of-file condition. Remember, this loop may terminate due to other failures as well. (We will discuss other error conditions later.)

There is another approach to detect the end-of-file condition. Note that we have used the following statement in Program :

if(finl.eof() !. 0) (exit(1);)

eof I is a member function of ion class. It returns a non-zero value if the end-of-file (EOF) condition is encountered. And a zero, otherwise. Therefore, the above statement terminates the program on reaching the end of the file.

Lab Exercise

```
#include <iostream.h>
#include <fstream.h>
#include <stdlib.h>            // for exit() function

int main()
{
          const  int SIZE = 80;
          char line[SIZE];

          ifstream fin1, fin2;        // create two input streams
          fin1.open("country");
          fin2.open("capital");

          for(int i=1; i<=10; i++)
          {
                 if(fin1.eof() != 0)
                 {
                        cout << "Exit from country \n";
                        exit(1);
                 }
          fin1.getline(line, SIZE);
          cout << "Capital of "<< line ;

          if(fin2.eof() != 0)
          {
                 cout << "Exit from capital\n";
                 exit(1);
          }

          fin2.getline(line,SIZE);
          cout << line << "\n";
    }
    return 0;
}
```

Lab Exercise

```
#include <iostream>
#include <fstream>
using namespace std;
int main () {
ifstream is("demo.txt");
  char c;
  while (is.get(c))
  cout << c;
  if (is.eof())
  cout << "EoF reached";
  else
  cout << "error reading";
  is.close();
  return 0;
}
```

Output

```
New file created
Process returned 0 (0x0)   execution time : 0.013 s
Press any key to continue.
```

Task

Write a program using C++ to demonstrate process of reading from file in C++.

Write a program using C++ to demonstrate process of writing in file.

Summary

- The C++ I/O system contains classes such as ifstream, ofstream and fatream to deal with file handling. These classes are derived from fstreambase class and are declared in a header file iostream.
- A file can be opened in two ways by using the constructor function of the class and using the member function open() of the class. While opening the file using constructor, we need to pass the desired filename as a parameter to the constructor.
- The open() function can be used to open multiple files that use the same stream object. The second argument of the open() function called file mode, specifies the purpose for which the file is opened.
- If we do not specify the second argument of the open() function, the default values specified in the prototype of these class member functions are used while opening the file. The default values are as follows:
  ios :: in – for ifstream functions, meaning-open for reading only.
  ios :: out – for ofstream functions, meaning-open for writing only.
- When a file is opened for writing only, a new file is created only if there is no file of that name. If a file by that name already exists, then its contents are deleted and the file is presented as a clean file.
- To open an existing file for updating without losing its original contents, we need to open it in an append mode.
- The (stream class does not provide a mode by default and therefore we must provide the mode explicitly when using an object of (stream class. We can specify more than one file modes using bitwise OR operator while opening a file.

Keywords

ofstream: This Stream class signifies the output file stream and is applied to create files for writing information to files

ifstream: This Stream class signifies the input file stream and is applied for reading information from files

fstream: This Stream class can be used for both read and write from/to files.

File: A file is a collection of related data stored in a particular area on the disk.

eof(): It returns non-zero when the end of file has been reached, otherwise it returns zero.

Self Assessment

1. C++ uses <iostream.h> directive because
A. C++ is an object oriented language
B. C++ is a markup language
C. C++ does not have any input/output facility
D. All of the above

2. The unformatted input functions are handled by
A. ostream class
B. instream class
C. istream class
D. bufStream class

3. The istream class defines the
A. Cin objects
B. Stream extraction operator for formatted input
C. Cout objects
D. Both Cin and Cout objects

4. iostream is a subclass of
A. istream
B. instream
C. ostream
D. Both istream and ostream

5. The class fstream is used for
A. High level stream processing
B. Low level stream processing
C. File stream Processing
D. All above

6. Which stream class is to only write on a file?
A. ofstream
B. ifstream
C. fstream
D. iostream

7. Which is correct syntax?
A. Myfile:open(“example.bin”,ios::out);
B. Myfile.open(“example.bin”,ios::out);
C. Myfile::open(“example.bin”,ios::out);
D. Myfile.open(“example.bin”,ios:out);

8. Which operator is used to insert the data into a file?

A. @

B. >

C. <<

D. None of above

9. Which of the following is the default mode of the opening using the ofstream class?

A. ios::in

B. ios::trunk

C. ios::out

D. ios::app

10. Which of the following is the default mode of the opening using the fstream class?

A. ios::in | ios::out

B. ios::trunk

C. ios::out

D. ios::in

11. "ios::app" causes all output to that file to be appended to the end. This value can be used only with file capable of output.

A. True

B. False

12. The close() function is used to close a file,closed by disconnecting with its streaming.

A. True

B. False

13. Which among following is correct syntax of closing a file?

A. myfile@close();

B. myfile.close();

C. myfile:close();

D. myfile$close();

14. Detection of end of file not possible in C++.

A. True

B. False

15. Which of these is the correct statement about eof() ?

A. Returns true if a file opens for reading has reached the next character.

B. Returns true if a file opens for reading has reached the next word.

C. Returns true if a file opens for reading has reached the end.

D. Returns true if a file opens for reading has reached the middle.

Answers for Self Assessment

1.	C	2.	C	3.	D	4.	D	5.	C
6.	A	7.	B	8.	C	9.	C	10.	A
11.	A	12.	A	13.	B	14.	A	15.	C

Review Questions

1. What do you mean by C++ streams?
2. What are the uses of files in computer system and how data can be write using C++.
3. What are the steps involved in using a file in a C++ program.
4. What is a file mode? Describe the various file mode options available.
5. Describe the various approaches by which we can detect end of file condition successfully.
6. Write a C++ program to demonstrate working of detection of end of file in C++.
7. What are the advantages of files?
8. How can we open a file? Explain with suitable example.
9. Write full process with suitable C++ program for create a new file.
10. Explain file opening process in C++.

Further Readings

E Balagurusamy; Object-oriented Programming with C++; Tata Mc Graw-Hill.

Herbert Schildt; The Complete Reference C++; Tata Mc Graw Hill.

Robert Lafore; Object-oriented Programming in Turbo C++; Galgotia.

Web Links

https://study.com/academy/lesson/practical-application-for-c-plus-plus-programming-working-with-files.html

https://www.codecademy.com/learn/learn-c-plus-plus

https://www.guru99.com/cpp-file-read-write-open.html